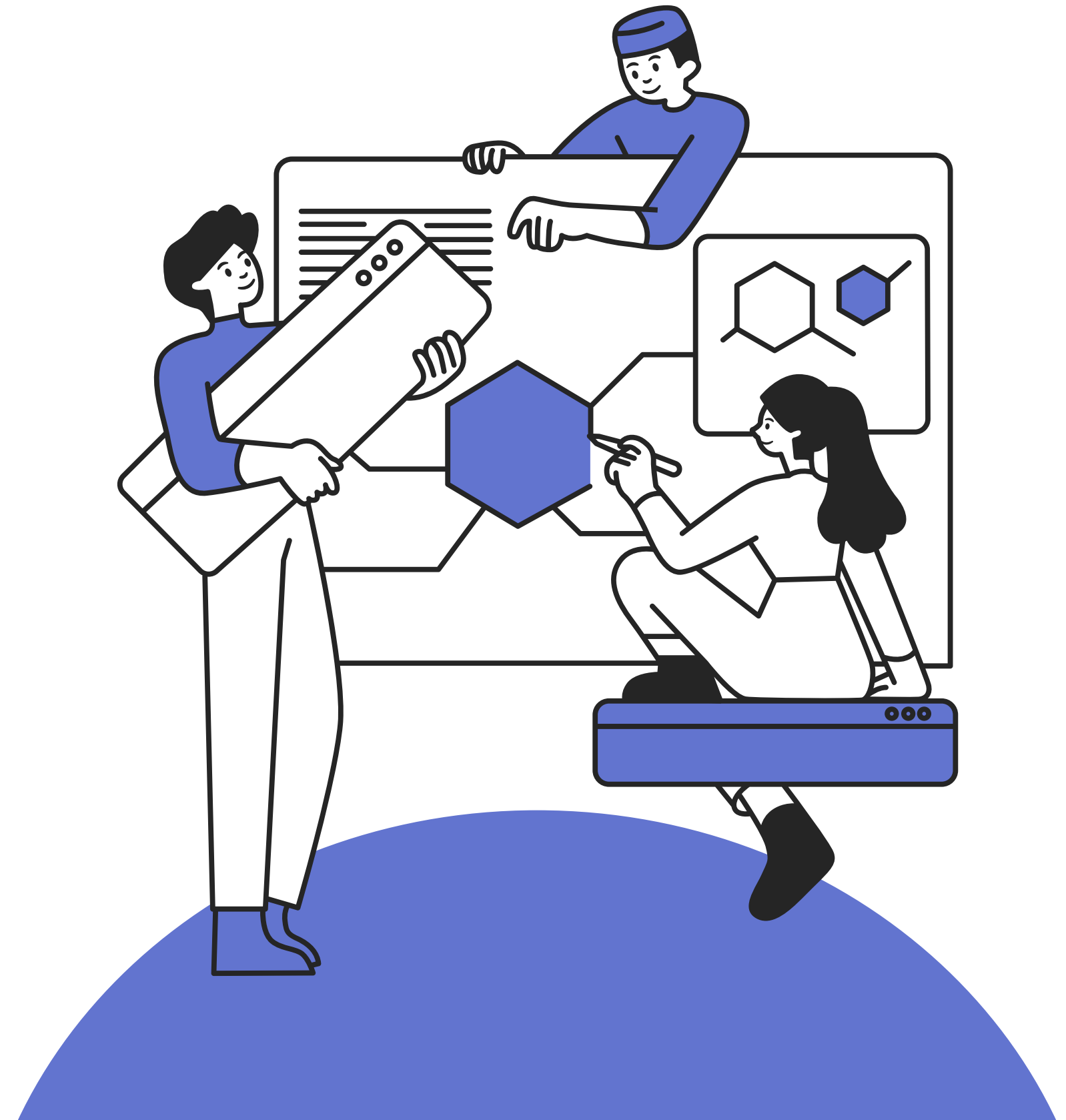


분할정보

이예린T

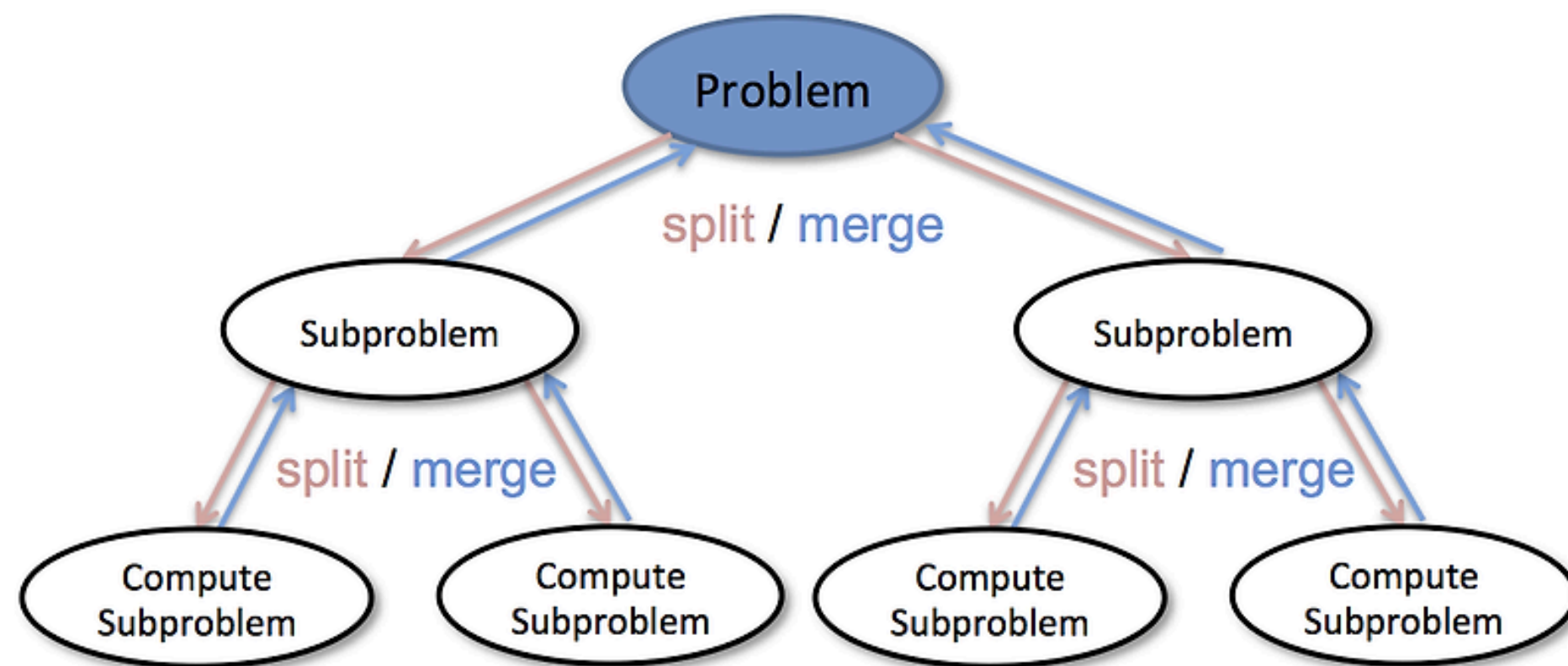


CONTENTS

- 1 분할정복
- 2 분할정복 대표 문제
- 3 모듈별 탐구
- 4 분할정복 정리

1 분할정보

하
일
이
산
더
미

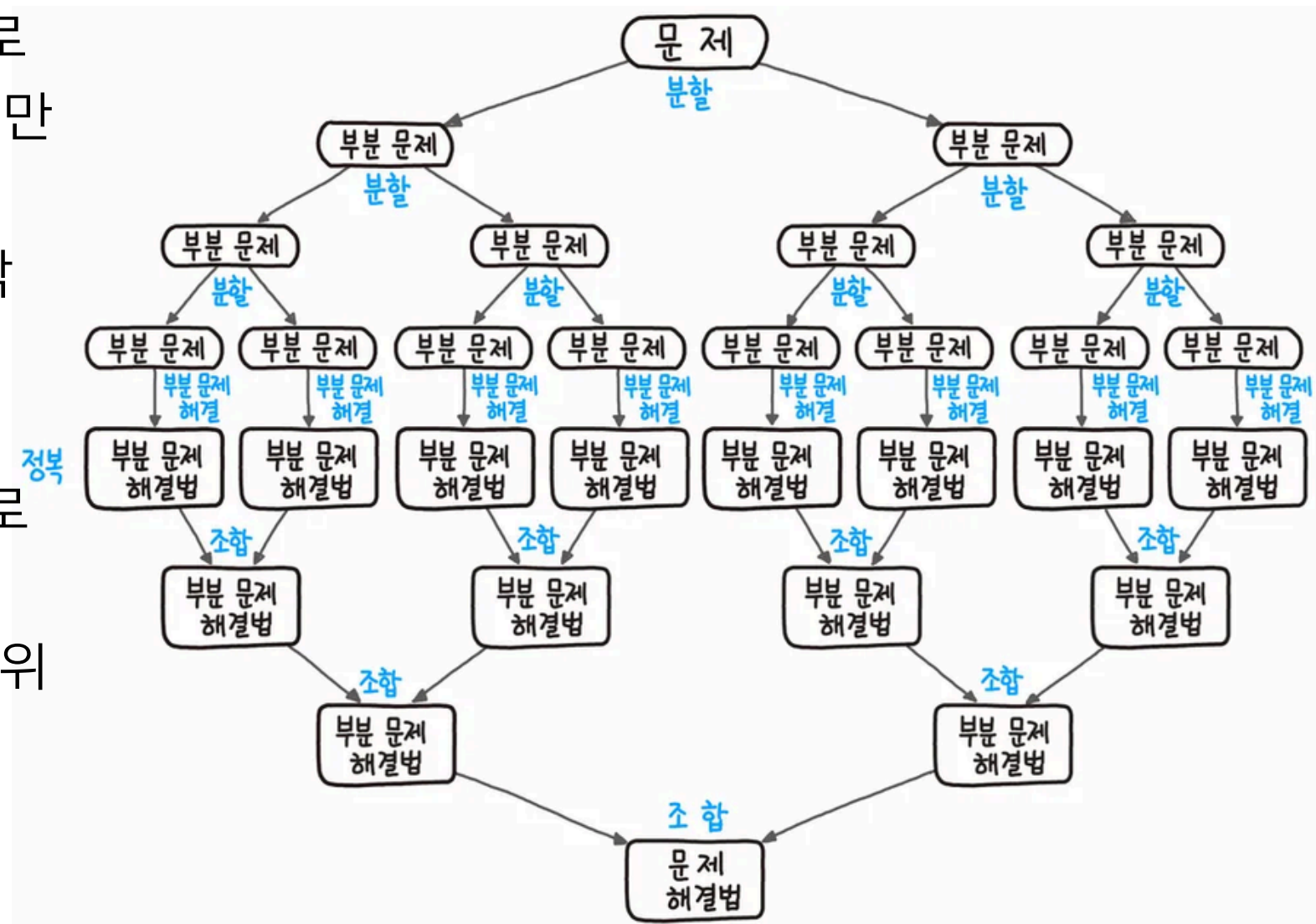


01

분할정복

Divide and Conquer

- 직접 해결할 수 있을 정도로 간단한 문제가 될 때까지 문제를 재귀적으로 쪼개나간 다음, 그 하위 문제의 결과들을 조합하여 원래 문제의 결과로 만들어 낸다.
- 문제를 더 이상 나눌 수 없을 때까지 나누고, 이렇게 나뉜 문제들을 각각 풀어 결국 전체 문제의 답을 얻는 기법
- 분할 정복 알고리즘 설계 요령
 - 1) 분할(Divide): 문제가 분할 가능한 경우 2개 이상의 하위 문제로 나눈다.
 - 2) 정복(Conquer): 하위 문제가 여전히 분할 가능한 상태라면 하위 집합에 대해 '분할'을 수행한다. 그렇지 않다면 하위 문제를 푼다.
 - 3) 결합(Combine): '정복' 단계를 통해 구한 답을 취합한다.
- 나뉘어진 각 하위 문제는 완전히 독립적이다.
- 대표적인 분할정복 문제 - 병합 정렬, 거듭제곱 연산, 피보나치 수



2 분할정복 대표 문제

01

거듭제곱 연산

$c^{**}n$

- 거듭제곱 연산: $c^{**}n$ 이라면, c 를 n 번 곱해야 한다.
- $c^{**}2 = c * c$
- $c^{**}3 = c * c * c$
- $c^{**}n = c * c * c \dots * c$
- 지수만큼의 시간복잡도가 필요하다. : $O(n)$

```
def power_naive(base, exp):  
    result = 1  
    for _ in range(exp):  
        result *= base  
    return result
```

01

거듭제곱 연산

$c^{**}n$

- 거듭제곱 연산: $c^{**}n$ 이라면, c 를 n 번 곱해야 한다.
- $c^{**}2 = c * c$
- $c^{**}3 = c * c * c$
- $c^{**}n = c * c * c \dots * c$
- 지수만큼의 시간복잡도가 필요하다. : $O(n)$
- 분할정복 알고리즘을 설계하면 $O(\log n)$ 으로 성능 향상시킬 수 있다.

$$\begin{aligned} 2^{11} &= 2^5 \times 2^5 \times 2^1 \\ 2^5 &= 2^2 \times 2^2 \times 2^1 \\ 2^2 &= 2^1 \times 2^1 \end{aligned}$$

$$C^8 = C^8$$

짝수

$$C^8 = C^4 \times C^4$$

$$C^8 = C^2 \times C^2 \times C^2 \times C^2$$

$$C^8 = C \times C \times C \times C \times C \times C \times C \times C$$

$$C^7 = C^7$$

홀수

$$C^7 = C^3 \times C^3 \times C$$

$$C^7 = (C \times C \times C) \times (C \times C \times C) \times C$$

$$C^n = \begin{cases} C^{n/2} C^{n/2} & (n \text{은 짝수}) \\ C^{(n-1)/2} C^{(n-1)/2} C & (n \text{은 홀수}) \end{cases}$$

01

거듭제곱 연산

$c^{**}n$

$$C^n = \begin{cases} C^{n/2} C^{n/2} & (n \text{은 짝수}) \\ C^{(n-1)/2} C^{(n-1)/2} C & (n \text{은 홀수}) \end{cases}$$

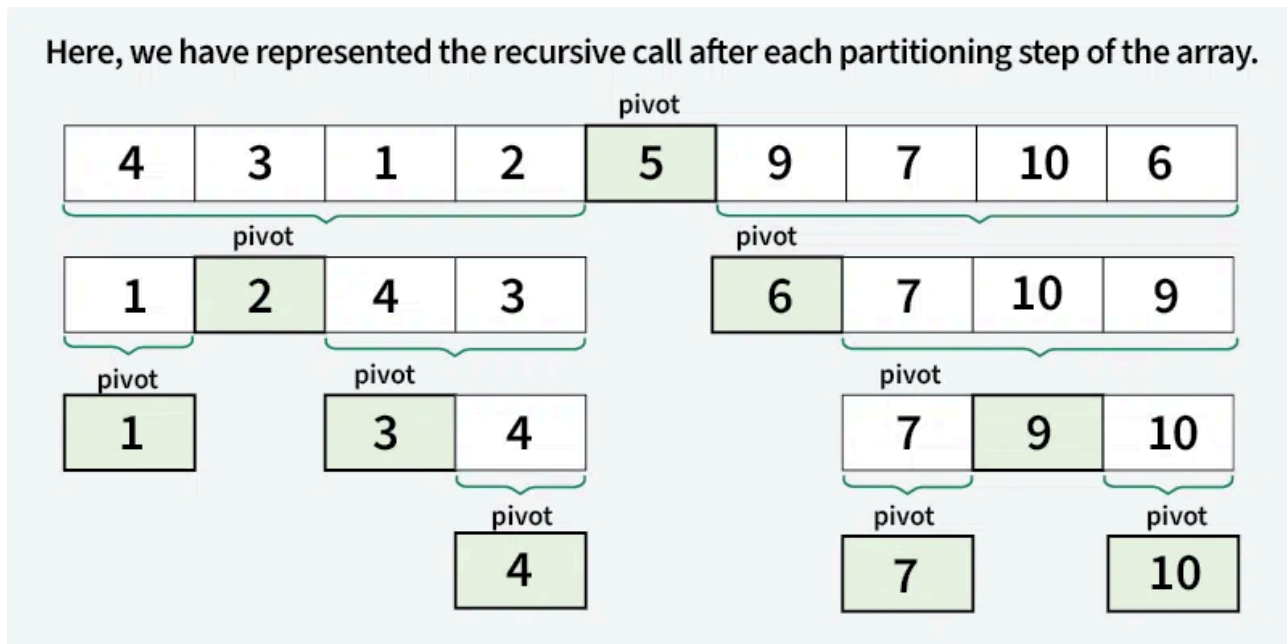
```
1 def power_dc(base, exp):
2     if exp == 0:
3         return 1
4     if exp == 1:
5         return base
6
7     half = power_dc(base, exp // 2)
8
9     if exp % 2 == 0:          # 지수가 짝수
10         return half * half
11 else:                        # 지수가 홀수
12     return half * half * base
```

02

퀵 정렬

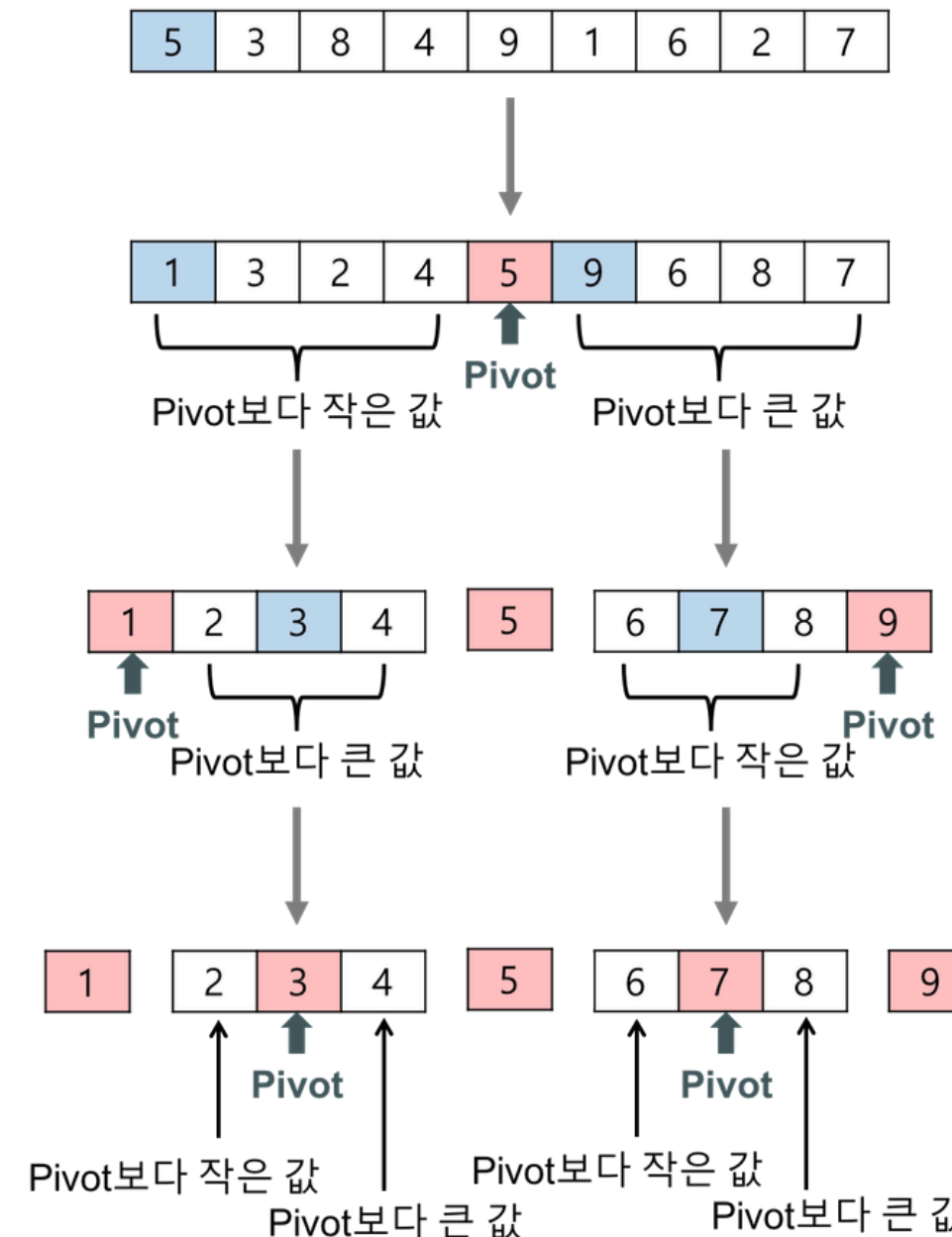
quick하게 해결하는 비결이 바로 분할정복

- 퀵 정렬: 임의의 기준 요소를 정하고 기준 요소보다 작은 요소들을 왼쪽으로, 큰 요소들을 오른쪽으로 옮긴다. 그리고 기준 요소에 의해 나뉜 양편의 요소들에 대하여 왼쪽/오른쪽 각각 분할을 수행한다. 이렇게 분할된 데이터들을 더 이상 분할할 수 없을 때까지 계속해서 분할한다.
- 분할: 피벗을 기준으로 배열을 피벗보다 작은 값, 피벗 값, 피벗보다 큰 값 세 그룹으로 나눈다.
- 정복: 나뉜 왼쪽과 오른쪽 그룹에 대해 퀵 정렬을 재귀적으로 수행한다.
- 결합: 정렬된 세 그룹을 하나의 리스트로 합친다.



초기상태

5	3	8	4	9	1	6	2	7
---	---	---	---	---	---	---	---	---



리스트의 크기가 0이나 1이 될 때까지 반복

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

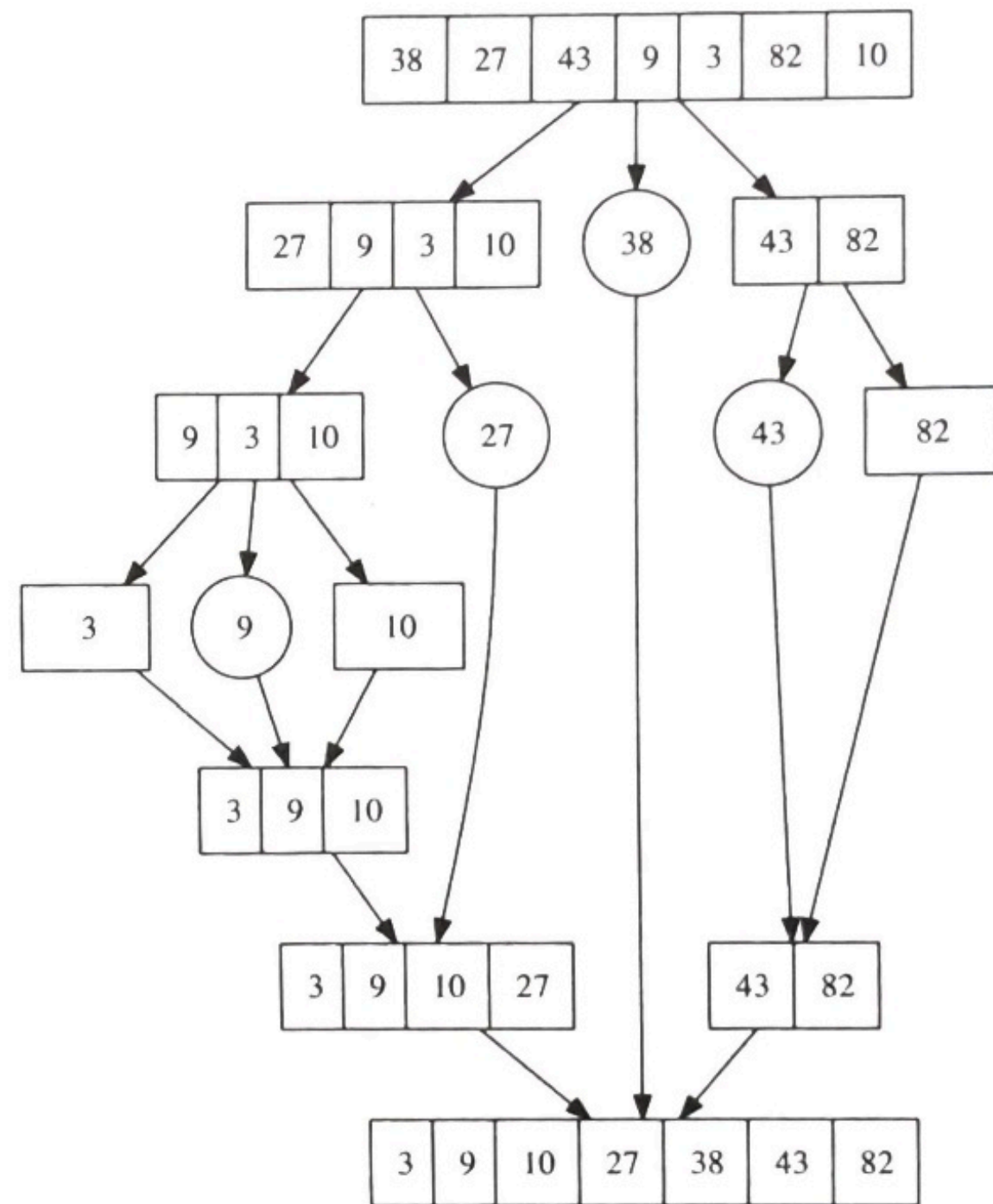
오름차순
완성상태

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

02

퀵 정렬

quick하게 해결하는 비결이 바로 분할정복



```

def quick_sort(array, start, end):
    if start >= end:
        return
    pivot = start #피벗 초기값은 첫번째 요소
    left = start+1
    right = end

    while left <= right:
        while left <= end and array[left] <= array[pivot]:
            left+=1
        while right > start and array[right] >= array[pivot]:
            right-=1
        if left>right: # 엇갈렸다면 작은 데이터와 피벗을 교체
            array[right], array[pivot] = array[pivot], array[right]
        else: # 엇갈리지 않았다면 작은 데이터와 큰 데이터를 교체
            array[left], array[right] = array[right], array[left]
        quick_sort(array, start, right-1)
        quick_sort(array, right+1, end)

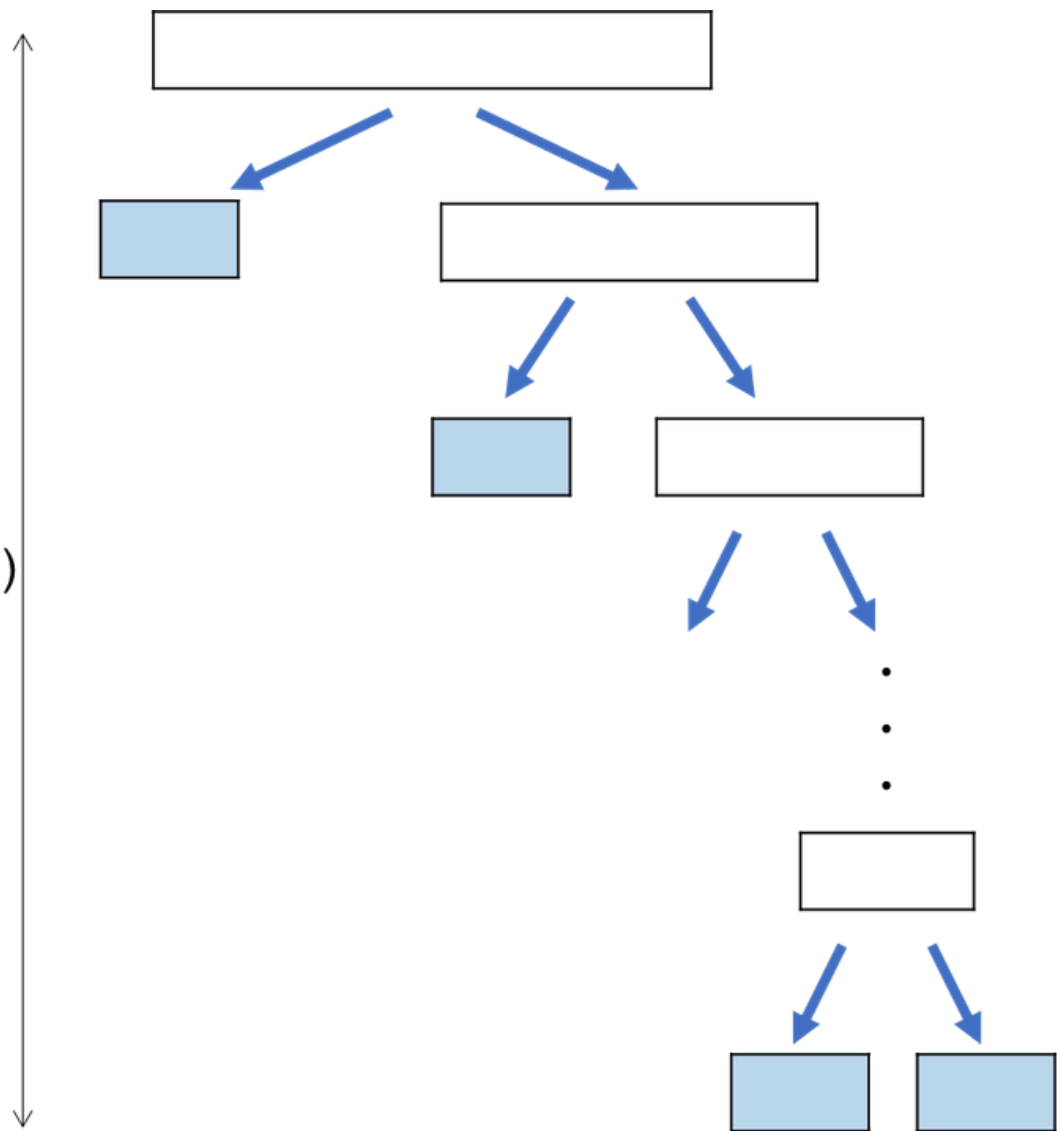
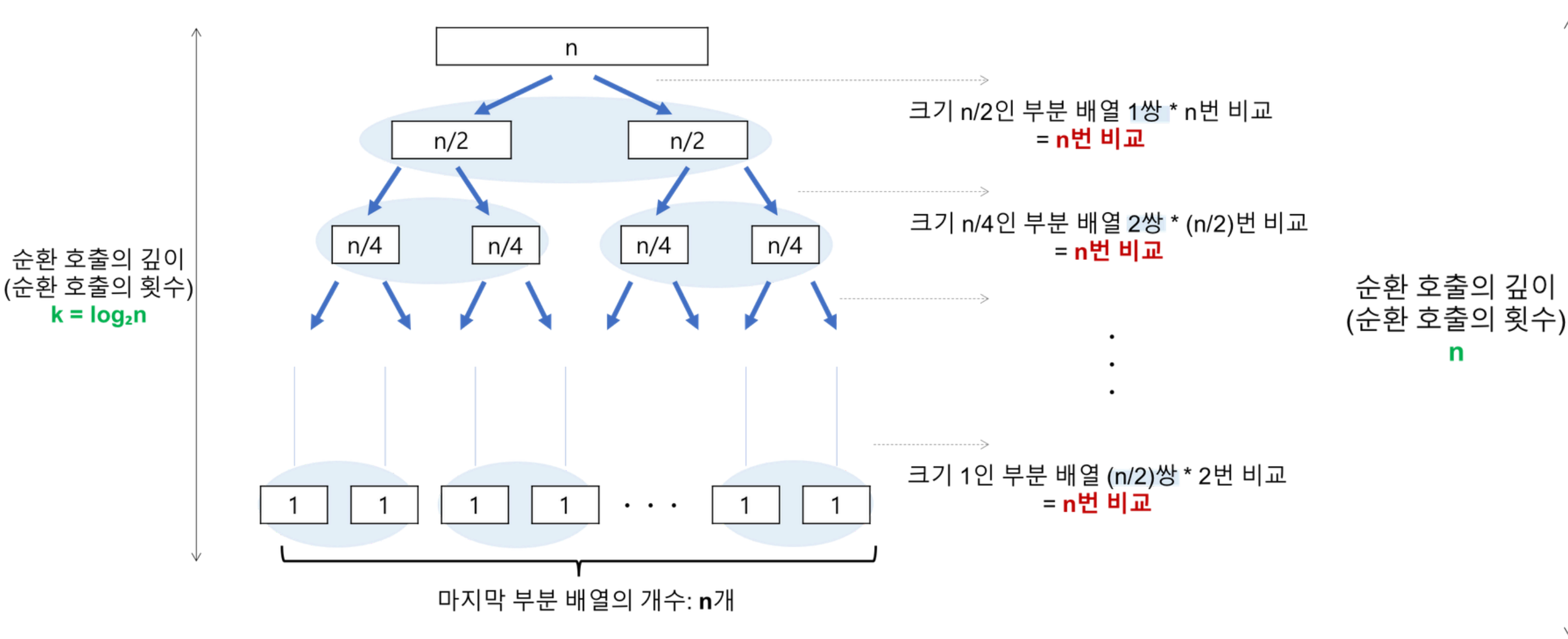
quick_sort(array, 0, len(array)-1)
    
```

02

퀵 정렬

quick하게 해결하는 비결이 바로 분할정복

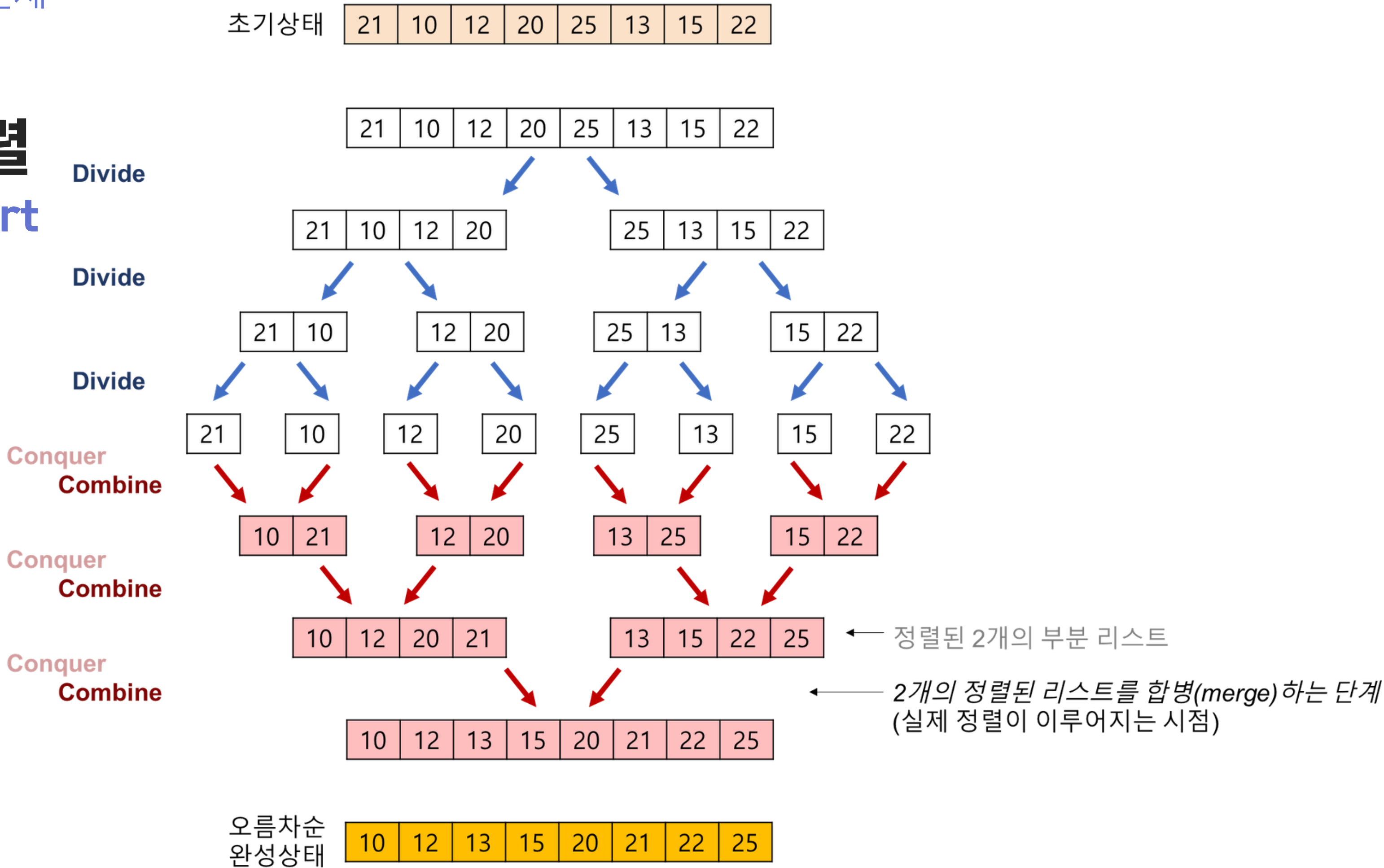
- 퀵 정렬의 시간복잡도: $O(n\log n) \sim O(n^2)$
- 피벗이 중간값일 때: $O(n\log n)$
- 피벗이 최솟값/최댓값일 때: $O(n^2)$
- 분할이 균등할수록 효율적이다.



03

병합 정렬

Merge Sort

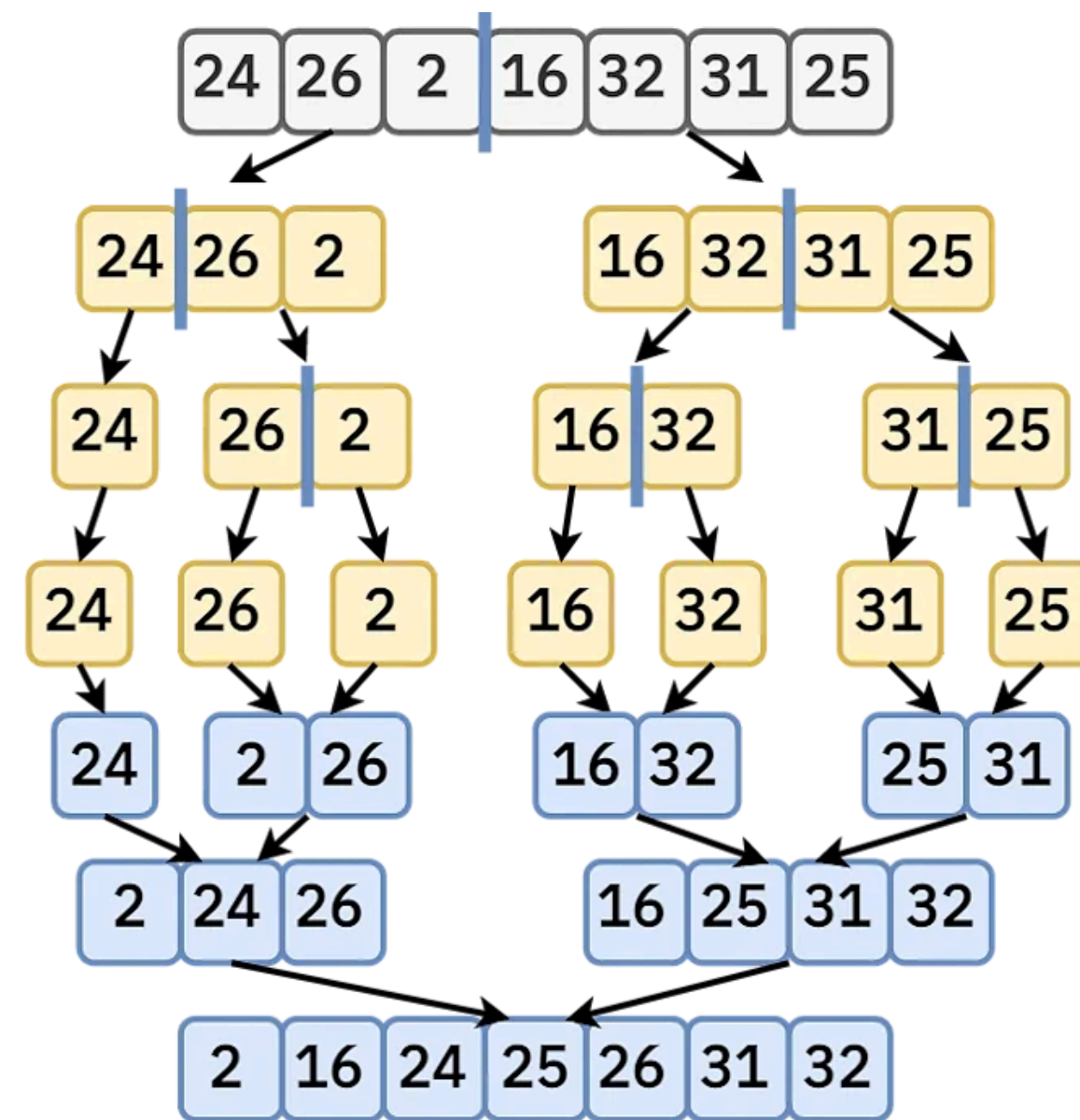


03

병합 정렬

Merge Sort

- 병합 정렬: 데이터를 나누고 합치면서 순서에 맞게 정렬함.
- 병합 정렬 알고리즘:
 - 1) 정렬할 데이터를 반으로 나눈다.
 - 2) 나뉜 하위 데이터의 크기가 2 이상이면 이 하위 데이터에 대해 1)을 반복한다.
 - 3) 동일 데이터에서 나뉜 하위 데이터 둘을 병합하여 원래대로 하나의 데이터로 만든다. 단, **병합할 때 데이터의 원소는 정렬 기준에 따라 정렬한다.**
 - 4) 데이터가 원래대로 모두 하나가 될 때까지 3)을 반복한다.

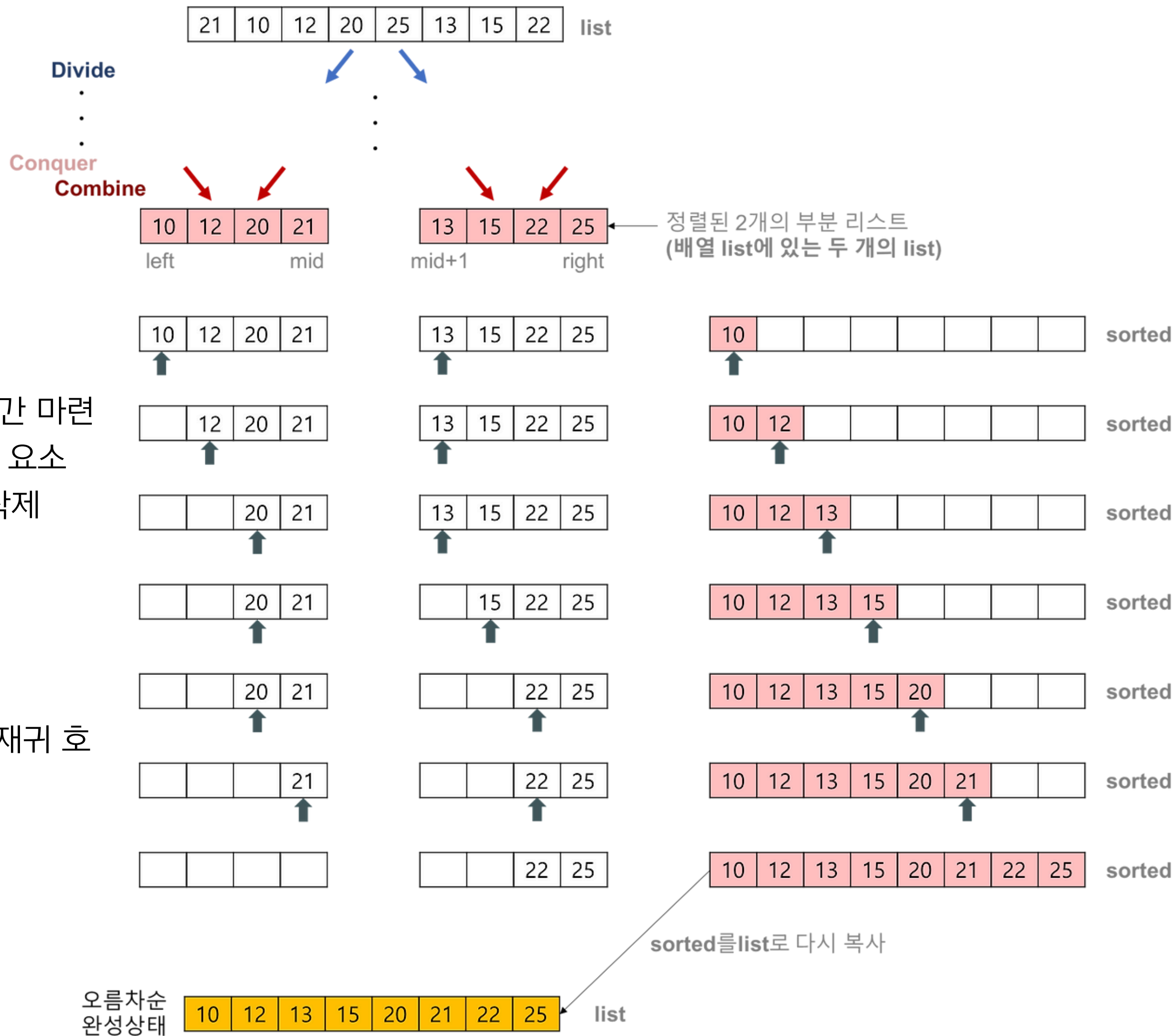


03

병합 정렬

Merge Sort

- 정렬하며 합치는 방법:
 - 두 데이터를 합한 것만큼 비어 있는 공간 마련
 - 두 데이터의 첫 요소 비교하여 작은 요소를 새 공간에 추가, 원래 데이터에서 삭제
 - 양쪽 데이터가 빌 때까지 2) 반복
- 구현:
 - 데이터를 반으로 나누고,
 - 반으로 나눈 데이터를 매개변수 삼아 재귀 호출하고,
 - 둘로 나눈 데이터를 다시 병합한다.



03

병합 정렬

Quick Sort VS Merge Sort

	퀵 정렬	병합 정렬
분할	피벗 기준 크기에 따라 불균등하게 나뉨	항상 절반으로 균등하게 나뉨
결합	단순 연결	두 배열을 비교하며 병합
최악 시간복잡도	$O(n^2)$	$O(n\log n)$

3 모듈별 탐구

01

이진탐색

- 정렬된 정수 배열에서 target을 찾되, target과 같은 값이 여러 개 있을 경우 가장 왼쪽 인덱스를 반환하시오.
- target이 없으면 -1을 반환한다.
- 두 가지 코드를 구현하시오.
 - 반복문으로 구현(재귀 호출 X)
 - 분할정복으로 구현(재귀 호출 O)

02

수식과 괄호 삽입

- 수식 문자열이 주어질 때, 괄호를 삽입할 수 있는 모든 경우의 결과값을 출력하시오.
- 단,
 - 입력은 한 자리 정수와 +, -, * 연산자로만 이루어진다
 - 연산자 우선순위는 무시하고, 괄호로 계산 순서를 완전히 결정한다
 - 결과값은 중복 제거 후 집합(set)으로 반환한다
 - 분할정복 구조로 구현할 것: 연산자를 기준으로 왼쪽/오른쪽을 재귀적으로 분할하고, 두 쪽의 결과를 결합할 것
 - 코드에 분할 / 정복 / 결합 단계를 주석으로 명시할 것
- 예) $2-1-1 \rightarrow [0, 2]$
- 예) $2*3-4*5 \rightarrow [-34, -14, -10, 10]$ #같은 결과값은 한 번만 출력되게 하시오.

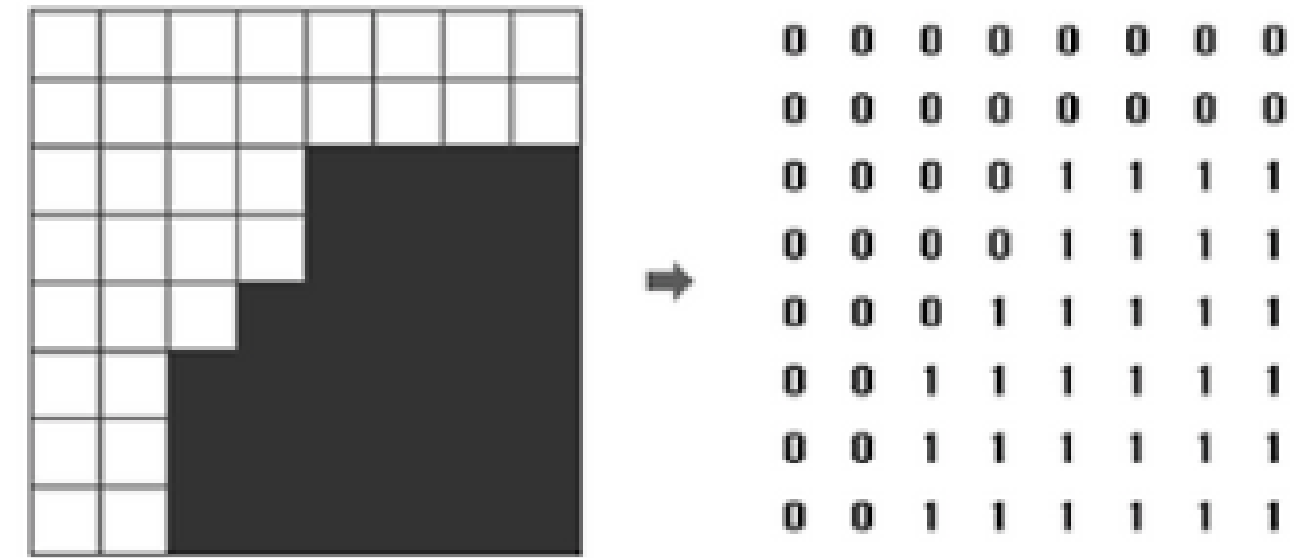
03

학급회장 찾기

- 선거 투표 결과가 리스트로 주어진다. 각 원소는 투표자가 찍은 후보의 번호고, 세종과고는 과반수 득표해야 회장으로 결정된다. 회장이 된 후보를 반환하시오. (단, 과반수 득표가 없을 때는 재투표가 필요하다고 출력한다.)
- 두 가지로 구현하시오.
 - Brute Force
 - Divide and Conquer
 - 분할정복 코드에는 코드에 분할/정복/결합 단계를 주석으로 표기할 것

04

쿼드 트리



- 흑백 영상을 압축하여 표현하는 데이터 구조로 쿼드 트리(Quad Tree)라는 방법이 있다. 흰 점을 나타내는 0과 검은 점을 나타내는 1로만 이루어진 영상(2차원 배열)에서 같은 숫자의 점 들이 한 곳에 많이 몰려있으면, 쿼드 트리에서는 이를 압축하여 간단히 표현할 수 있다.
 - 주어진 영상이 모두 0으로만 되어 있으면 압축 결과는 0이 되고, 모두 1로만 되어 있으면 압축 결과는 1이 된다. 만약 0과 1이 섞여 있으면 전체를 한 번에 나타내지를 못하고, 왼쪽 위, 오른쪽 위, 왼쪽 아래, 오른쪽 아래, 이렇게 4개의 영상으로 나누어 압축하게 되며, 이 4개의 영역을 압축한 결과를 차례대로 괄호 안에 묶어서 표현한다.
 - 단, 분할정복 구조로 구현할 것
 - 코드에 분할 / 정복 / 결합 단계를 주석으로 명시할 것
 - 압축률을 출력할 것
- 위 그림에서 왼쪽의 영상은 오른쪽의 배열과 같이 숫자로 주어지며, 이 영상을 쿼드 트리 구조를 이용하여 압축하면 (0(0011)(0(0111)01)1)로 표현된다. (압축률: 32.8%)
- 입력: 첫째 줄에는 영상의 크기를 나타내는 숫자 N 이 주어진다. N 은 언제나 2의 제곱수로 주어지며, $1 \leq N \leq 64$ 의 범위를 가진다. 두 번째 줄부터는 길이 N의 문자열이 N개 들어온다. 각 문자열은 0 또는 1의 숫자로 이루어져 있으며, 영상의 각 점들을 나타낸다.
- 출력: 영상을 압축한 결과와 압축률을 출력한다.

05

최근접 점 쌍

- 2차원 평면 위의 점 n 개가 주어질 때, 가장 가까운 두 점 사이의 거리를 구하시오. 분할정복으로 구현할 것.
- 입력
 - 첫째 줄: 점의 개수 n
 - 둘째 줄부터: x y 좌표 (실수)
- 출력
 - 가장 가까운 두 점의 거리 (소수점 둘째 자리까지)

06

광고 가성비 챙기기

- 동영상 플랫폼에서 광고를 삽입할 구간을 정해야 한다. 영상 전체에서 가장 많은 시청자가 동시에 보고 있는 연속 구간 M초를 찾아 광고를 삽입하려 한다. N명의 시청자가 각각 시청을 시작한 시각과 끝낸 시각이 주어진다. 길이가 M초인 구간 중 누적 시청자 수(시청자 수 × 초)가 최대인 구간의 시작 시각을 구하시오. 분할정복으로 구현할 것.
- 입력
 - 총 영상 길이 total, 광고 길이 M
 - 시청 로그: [(시작초, 끝초), ...]
- 출력
 - 초별 시청자 수
 - 광고 시작 시각 (초) (동점이면 가장 이른 시각)
- 예)

total = 10, M = 3 logs = [(1,5), (2,6), (3,7), (4,8), (5,9)] #입력

초별 시청자 수: [0,1,2,3,4,4,3,2,1,0] #출력

구간별 누적: 0~2: 0+1+2 = 3 1~3: 1+2+3 = 6 2~4: 2+3+4 = 9

3~5: 3+4+4 = 11 ← 최대 4~6: 4+4+3 = 11 ...

광고 시작: 3 (동점이면 가장 이른 시각) #출력

4 분할정복 정리

01

분할, 정복, 결합

- 1) 분할(Divide): 문제가 분할 가능한 경우 2개 이상의 하위 문제로 나눈다. → 어떻게 쪼갤까
- 2) 정복(Conquer): 하위 문제가 여전히 분할 가능한 상태라면 하위 집합에 대해 ‘분할’을 수행한다. 그렇지 않다면 하위 문제를 푼다. → 쪼갬 걸 풀기 (= 재귀적으로 똑같이)
- 3) 결합(Combine): ‘정복’ 단계를 통해 구한 답을 취합한다. → 푼 내용을 합치기

02

이진탐색의 구현

결합 단계

```
def binary_search_dc(arr, target):  
  
    def search(left, right, answer):  
        if left > right:  
            return answer  
  
        mid = (left + right) // 2  
  
        if arr[mid] == target:  
            return search(left, mid - 1, mid)  
  
        elif arr[mid] > target:  
            return search(left, mid - 1, answer)  
  
        else:  
            return search(mid + 1, right, answer)  
  
    return search(0, len(arr) - 1, -1)
```

```
def binary_search_leftmost(array, target, start, end):  
    if start > end:  
        return -1  
  
    mid = (start + end) // 2  
  
    if array[mid] == target:  
        left_result = binary_search_leftmost(array, target, start, mid - 1)  
        return left_result if left_result != -1 else mid  
  
    elif array[mid] > target:  
        return binary_search_leftmost(array, target, start, mid - 1)  
    else:  
        return binary_search_leftmost(array, target, mid + 1, end)
```


03

최대 부분합 문제

maximum subarray

- 연속된 배열에서 합이 최대가 되는 연속 부분 합을 찾으시오.
- 여러 해결 방법이 있다.

예제 입력 1 복사

```
10
10 -4 3 1 5 6 -35 12 21 -1
```

예제 출력 1 복사

```
33
```

예제 입력 2 복사

```
10
2 1 -4 3 4 -4 6 5 -5 1
```

예제 출력 2 복사

```
14
```

예제 입력 3 복사

```
5
-1 -2 -3 -4 -5
```

예제 출력 3 복사

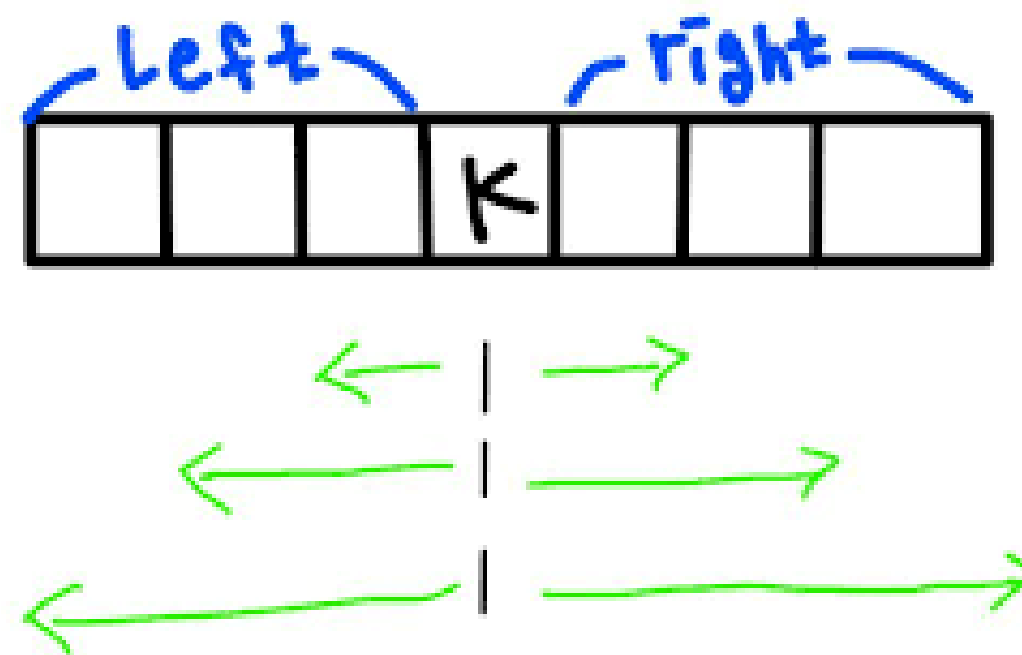
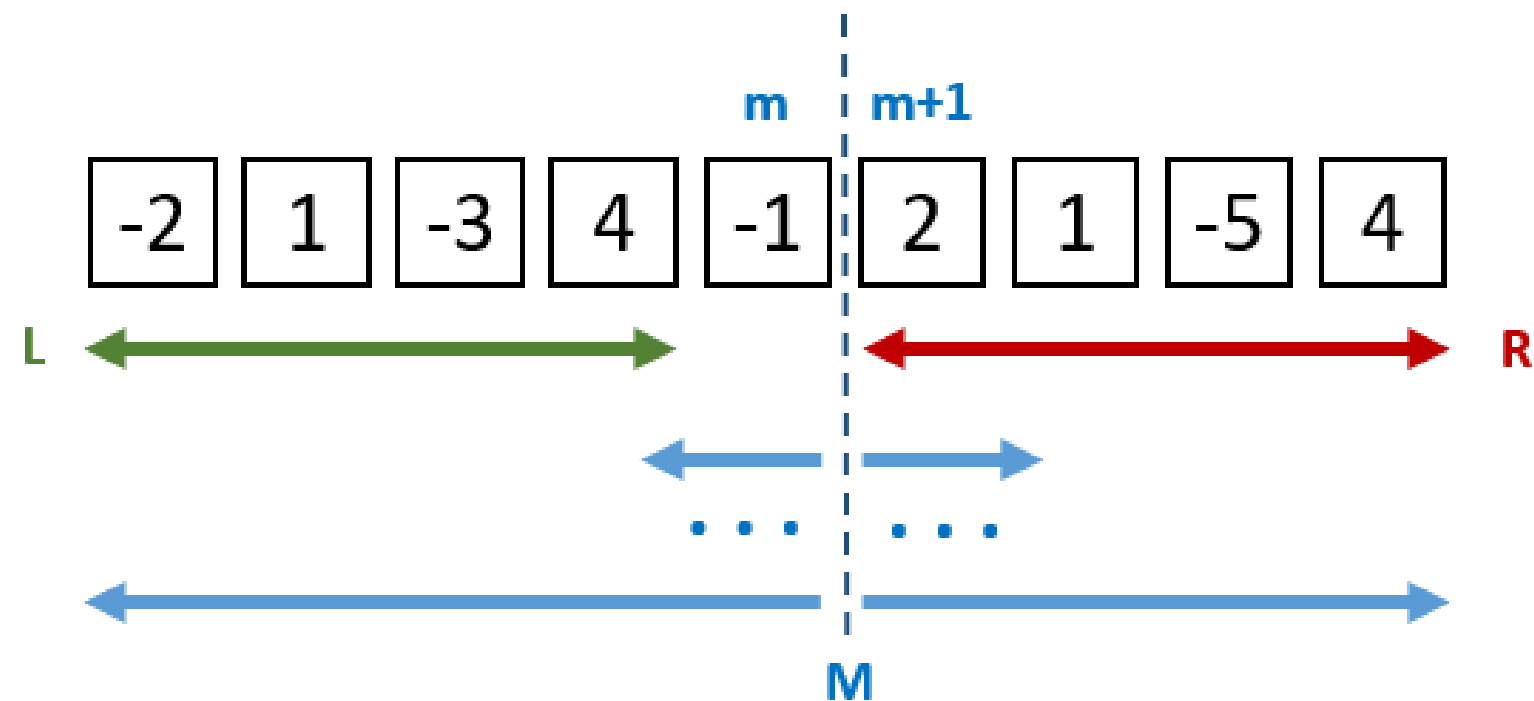
```
-1
```

03

최대 부분합 문제

maximum subarray

- 연속된 배열에서 합이 최대가 되는 연속 부분 합을 찾으시오.
- 10, -4, 3, 1, 5, 6, -35, 12, 21, -1
- 2, 1, -4, 3, 4, -4, 6, 5, -5, 1



03

최대 부분합 문제

maximum subarray

```

1 def max_crossing_sum(nums, left, mid, right):
2     left_sum = float('-inf')
3     cur = 0
4     for i in range(mid, left - 1, -1):
5         cur += nums[i]
6         left_sum = max(left_sum, cur)
7
8     right_sum = float('-inf')
9     cur = 0
10    for i in range(mid + 1, right + 1):
11        cur += nums[i]
12        right_sum = max(right_sum, cur)
13
14    return left_sum + right_sum

```

```

17 def max_subarray_dc(nums, left, right):
18     if left == right:
19         return nums[left]
20
21     # — 1단계: 분할 (Divide) —————
22     mid = (left + right) // 2
23     # 배열을 [left..mid] / [mid+1..right] 로 나눔
24
25     # — 2단계: 정복 (Conquer) —————
26     left_max = max_subarray_dc(nums, left, mid)      # 왼쪽 재귀
27     right_max = max_subarray_dc(nums, mid + 1, right) # 오른쪽 재귀
28
29     # — 3단계: 결합 (Combine) —————
30     cross_max = max_crossing_sum(nums, left, mid, right) # 중간 걸치는 경우
31     return max(left_max, right_max, cross_max)          # 셋 중 최댓값 반환
32
33
34 nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
35 print(max_subarray_dc(nums, 0, len(nums) - 1)) # 6

```

04

Decrease and Conquer

축소정복

- Decrease and conquer: 문제를 더 작은 문제로 쪼개나가며 축소시킨 문제를 해결하는데, 일정 크기만큼 문제를 줄여 나가는 전략이다.
- Decrease에 divide를 포함한다.
 - decrease by a constant: 상수만큼 줄이며 문제를 축소(1씩 감소)
 - decrease by a constant factor: 상수 인자만큼 문제를 축소. 즉, 문제를 나누어(divide) 축소
 - variable size decrease: 문제 축소 시 크기가 정해져 있지 않은 방식

04

Decrease and Conquer

축소정복

- decrease by a constant: 상수만큼 줄이며 문제를 축소(1씩 감소)
- 예시: 삽입 정렬
 - n 개의 원소가 있을 때, n 번째 원소를 나머지 $n-1$ 개의 원소로 이뤄진 이미 정렬된 배열에 삽입
 - 문제의 크기가 $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots$ (1씩 감소한다.)



Q&A
